Reporte de experimentos — MLflow Project y tracking avanzado

TC5061 · Operaciones de aprendizaje automático · Segunda entrega del proyecto integrador (MiBici)

Rolando Calderón Carballo — A01840537 · 24 de octubre de 2026

Entrega individual: por acuerdo del equipo, cada integrante continúa el proyecto por separado. Este trabajo parte del repositorio del Equipo 32 (primera entrega) y todo lo reportado aquí fue desarrollado y ejecutado individualmente en el repositorio indicado abajo.

1. Repositorio y comandos de reproducción

Repositorio: https://github.com/a01840537/TC5061-Proyecto-Equipo32-rolando

El repositorio es un MLflow Project: el entorno está declarado en python_env.yaml (versiones fijadas en requirements.txt) y toda la entrega se ejecuta con mlflow run, sin notebooks ni pasos manuales. En un clon limpio, siguiendo el README:

```
mlflow server --backend-store-uri sqlite:///mlflow.db --port 5000   # terminal dedicada
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
mlflow run . -e ingest --env-manager local --experiment-name mibici-equipo32
mlflow run . -e train -P question=p1 -P model=poisson --env-manager local --experiment-name mibici-equipo32
mlflow run . -e search -P question=p2 -P model=hgb --env-manager local --experiment-name mibici-equipo32
mlflow run . -e register -P question=p1 --env-manager local
mlflow run . -e verify -P question=p1 --env-manager local
```

Puntos de entrada del MLproject: ingest (ingesta reproducible con manifiesto de versión de datos), train (pregunta, modelo e hiperparámetros; el valor -1 delega en los defaults de config.py), search (búsqueda con nested runs), register (Model Registry) y verify (carga por alias). Todas las corridas registran como etiquetas el commit de Git y la versión de los datos (data_version 7aa5e436d887, 4,532,032 viajes de ene–dic 2025), lo que liga cada resultado al código y a los datos exactos que lo produjeron. Se mantuvieron la partición temporal y las métricas de la entrega anterior, por lo que la comparación contra los baselines es válida.

2. Comparación de experimentos por pregunta

Todas las corridas viven en el experimento único mibici-equipo32, etiquetadas con question, model_family y stage_of_work (baseline / candidato / search) para filtrar en la interfaz. Cada pregunta tiene su baseline, al menos cuatro corridas candidatas y una búsqueda de hiperparámetros organizada con nested runs (una corrida padre y una hija por combinación). La fila sombreada es el modelo elegido.

P1 — Demanda de viajes por estación y día (regresión)

Corrida	Hiperparámetros	val_mae	test_mae	test_wape	test_r2
p1_naive (baseline)	lag 7 días	6.232	7.219	0.234	0.854
p1_poisson	alpha=1e-3, log1p+OHE	5.104	5.410	0.175	0.923
p1_hgb_150_0.1	iter=150, lr=0.1	5.285	6.293	0.204	0.895
p1_hgb_300_0.06	iter=300, lr=0.06	5.260	6.311	0.204	0.897
p1_hgb_500_0.03	iter=500, lr=0.03	5.261	6.304	0.204	0.897
p1_hgb_300_0.06_d6	iter=300, lr=0.06, depth=6	5.286	6.267	0.203	0.899

Tabla 1. P1: baseline ingenuo estacional vs. Poisson lineal vs. búsqueda HGB (pérdida Poisson).

P2 — Riesgo de quedarse sin bicicletas o anclajes (clasificación)

Corrida	Hiperparámetros	val_f1	test_f1	test_recall	test_precision	test_pr_auc
p2_logreg (baseline)	logreg balanceada	0.415	0.365	0.454	0.305	0.351
p2_hgb_100_0.1	iter=100, lr=0.1	0.465	0.396	0.478	0.338	0.519
p2_hgb_200_0.1	iter=200, lr=0.1	0.468	0.402	0.459	0.358	0.504
p2_hgb_200_0.05_d8	iter=200, lr=0.05, depth=8	0.467	0.397	0.472	0.343	0.515
p2_hgb_400_0.05	iter=400, lr=0.05	0.468	0.402	0.470	0.351	0.515

Tabla 2. P2: regresión logística balanceada vs. búsqueda HGB. Umbral de decisión elegido en validación.

P3 — Grupos de estaciones por patrón de uso (clustering)

Corrida	train_silhouette	test_silhouette	estabilidad (ARI)	Davies-Bouldin
p3_kmeans2 (baseline)	0.198	0.186	0.726	1.772
p3_kmeans_k3	0.177	0.161	0.761	1.818
p3_kmeans_k4	0.186	0.167	0.793	1.584
p3_kmeans_k5	0.151	0.123	0.697	1.720
p3_kmeans_k6	0.147	0.117	0.719	1.781
p3_agglo_k4 (ward)	0.109	—	—	2.000

Tabla 3. P3: KMeans k=2 (baseline) vs. barrido de k con nested runs y aglomerativo (ward).

3. Análisis y justificación del modelo elegido

P1 — Poisson lineal. El modelo lineal con enlace log superó a las cuatro variantes de gradient boosting (test_mae 5.41 vs. 6.27–6.31) y mejoró 25% al baseline ingenuo (7.22). Dos razones: con rezagos y medias móviles bien construidos la relación con la demanda es casi multiplicativa, que es exactamente lo que el enlace log captura; y el boosting muestra brecha val→test (5.26 → 6.27+), señal de sobreajuste al periodo de validación. A igualdad aproximada de desempeño, el lineal gana también en costo (entrena en segundos, inferencia trivial) y en estabilidad. Es un recordatorio útil: el modelo más complejo no es el mejor por default.

P2 — HistGradientBoosting (iter=200, lr=0.1). Mejor test_f1 (0.402 vs. 0.365 del baseline, +10%) con precision y recall más balanceados. La mejora de fondo está en el PR-AUC: 0.35 → 0.50 (+43%), es decir, el modelo ordena mucho mejor el riesgo independientemente del umbral

— en operación, el equipo de rebalanceo puede recortar la lista al tamaño de su capacidad sin perder calidad. La métrica ligada a la decisión de negocio es el recall con precision de piso (cada estación vacía no detectada son viajes perdidos; cada falsa alarma, una visita en vano), y el f1 resume ese balance. Las cuatro variantes de la búsqueda quedaron en un rango estrecho (0.396–0.402), lo que da confianza en la estabilidad del resultado.

P3 — KMeans k=4. Aquí la selección exige criterio y no solo la métrica: el baseline k=2 tiene mayor silhouette de prueba (0.186 vs. 0.167), pero dos grupos (centro vs. periferia) no aportan nada accionable a planeación urbana. Entre los candidatos, k=4 domina: mejor silhouette de prueba del barrido, mejor Davies-Bouldin (1.58) y la mayor estabilidad temporal (ARI 0.79 entre los grupos de entrenamiento y los de oct–dic), y produce perfiles interpretables (intensidad alta, uso de fin de semana, origen/destino de commute). El aglomerativo quedó claramente por debajo (silhouette 0.109) y no permite asignar estaciones nuevas sin reajustar. La silhouette absoluta (~0.17) es baja, como es típico en datos de comportamiento urbano: los grupos son regiones de un continuo, no islas — se documenta como limitación.

4. Model Registry y ciclo de vida

El mejor modelo de cada pregunta quedó registrado con nombre claro y descripción con su linaje completo (corrida de origen, métrica de selección, commit de Git y data_version):

Modelo registrado	Champion (producción)	Métrica de selección
mibici-equipo32-p1-demanda	p1_poisson (v1)	test_mae = 5.410
mibici-equipo32-p2-riesgo	p2_hgb_200_0.1 (v1)	test_f1 = 0.402
mibici-equipo32-p3-grupos	p3_kmeans_k4 (v1)	test_silhouette = 0.167 (+ ARI 0.79)

Tabla 4. Modelos en el Registry con su versión en producción.

Esquema elegido: aliases. Se usó el esquema vigente de aliases de MLflow en lugar de las etapas clásicas (Staging/Production), por tres razones: las etapas están deprecadas en MLflow 3; los aliases permiten roles explícitos sin transiciones destructivas — champion apunta a la versión en producción y challenger a la segunda mejor, lista para un A/B o un reemplazo controlado —; y el consumidor carga el modelo por un nombre estable sin conocer la versión: models:/mibici-equipo32-p1-demanda@champion. La verificación del paso 8 se ejecutó con el punto de entrada verify, que carga cada champion por alias y reporta clase, versión y corrida de origen (captura en la sección 5).

5. Evidencias (capturas de MLflow)

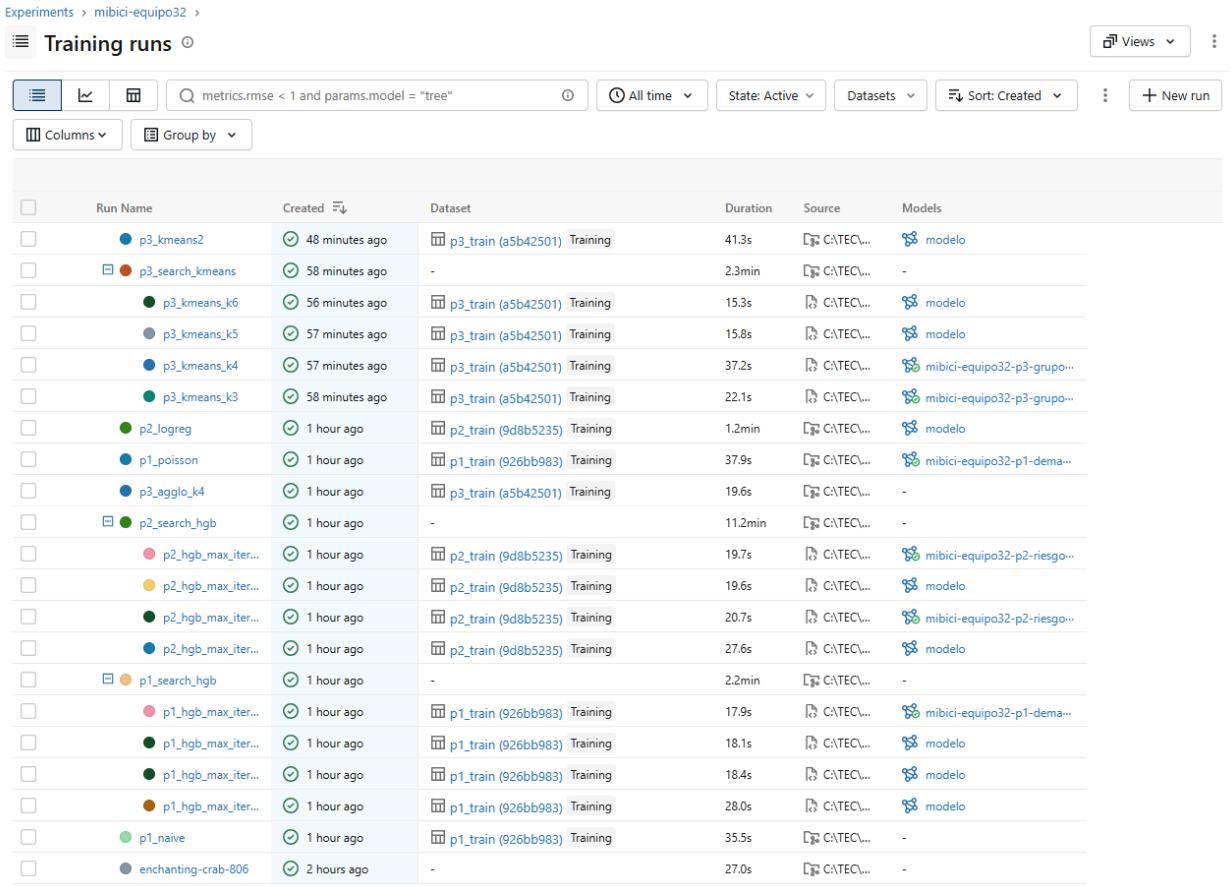


Figura 1. Búsqueda de hiperparámetros con nested runs en el experimento mibici-equipo32.



Figura 2. Artefactos de la corrida candidata p2_hgb_max_iter200_learning_rate0.1 organizados en subcarpetas: plots/ (matriz de confusión y curva PR), data/ (manifiesto con la versión de los datos) y config/ (configuración de la corrida). El modelo con firma queda ligado al run como Logged Model, del cual proviene la versión @champion mostrada en la Figura 3.

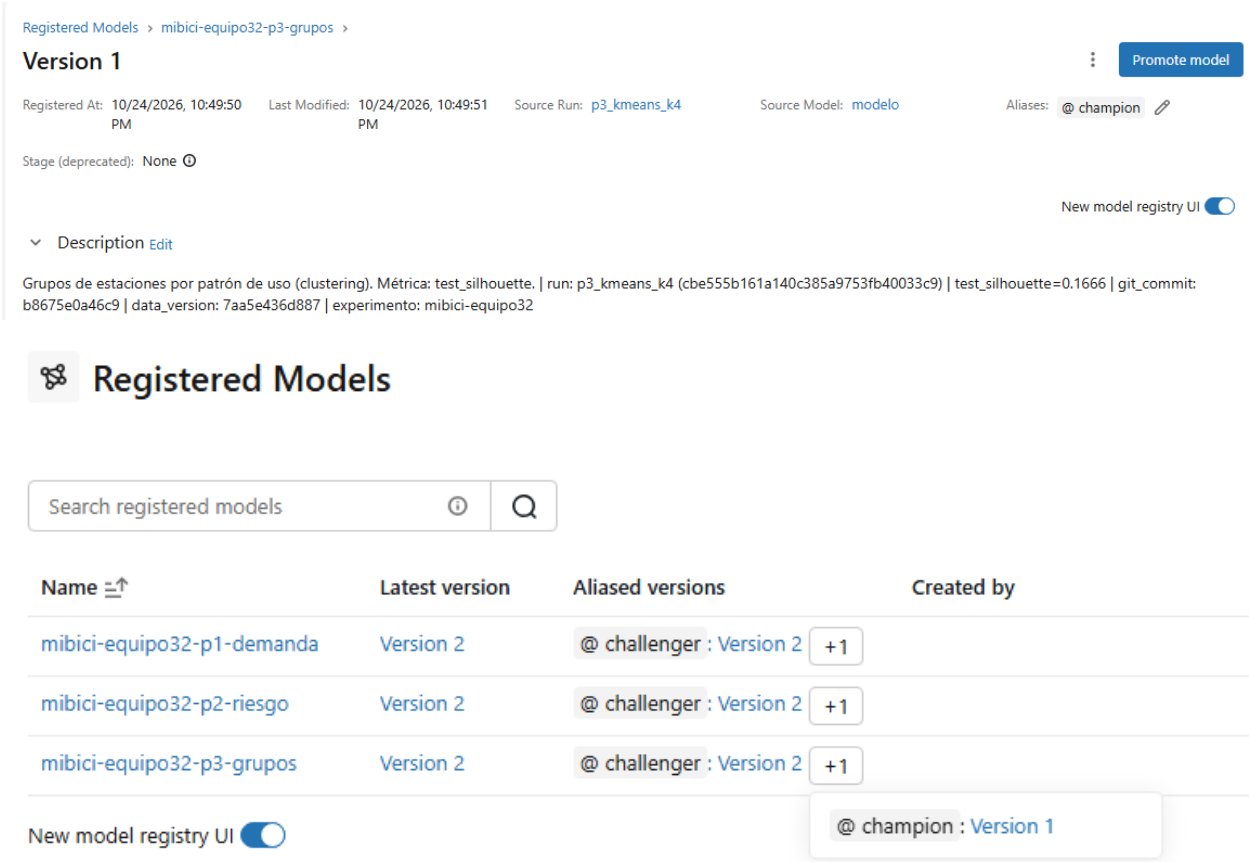


Figura 3. Model Registry: los tres modelos con versión en producción vía alias champion.

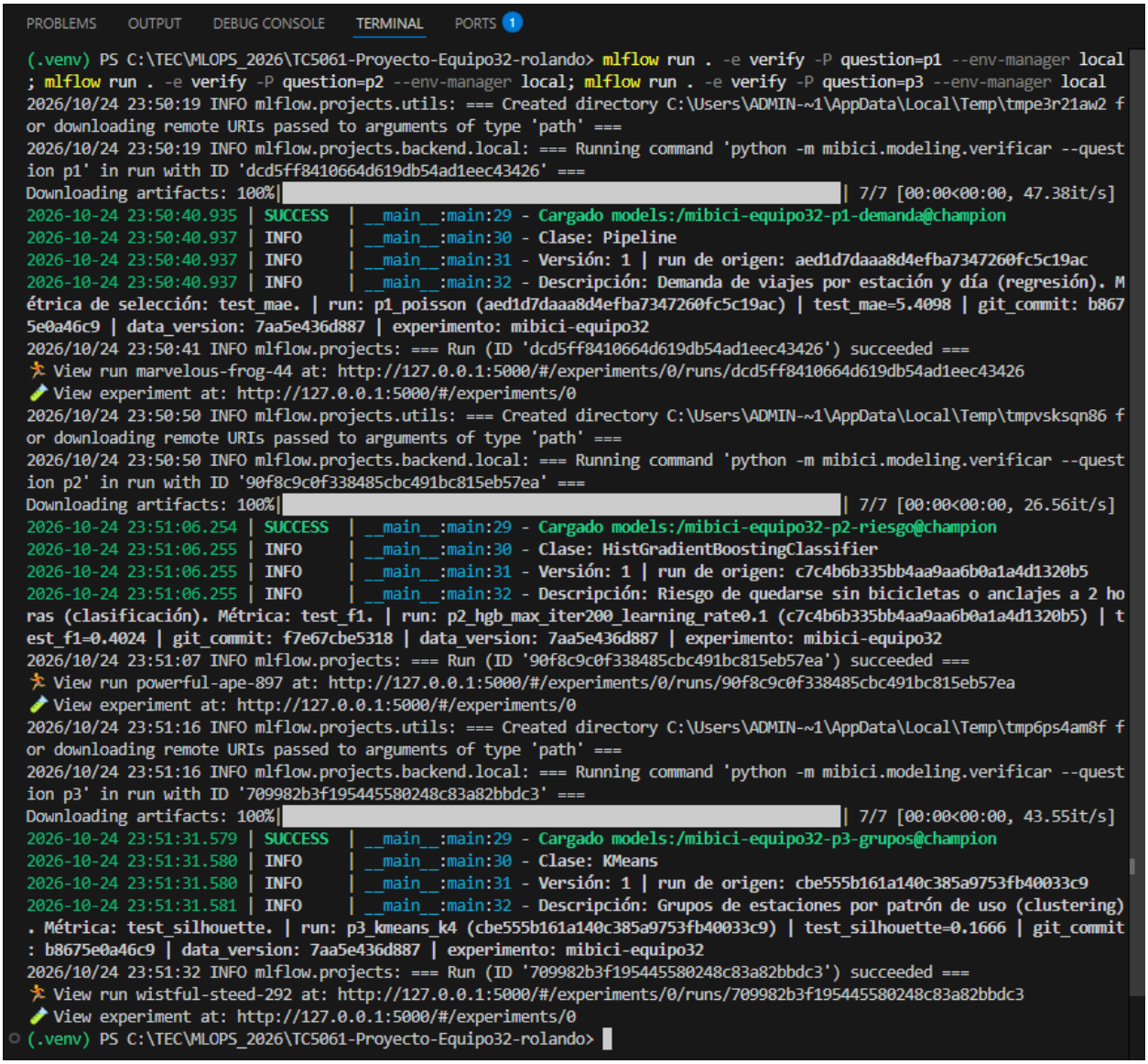


Figura 4. Verificación de carga: models:/<nombre>@champion para P1, P2 y P3.

6. Aprendizajes operativos

Dos incidentes reales de la ejecución valen como lecciones de MLOps. Primero: MLflow Project registra como parámetros del run los argumentos del punto de entrada, y chocan con los hiperparámetros reales del modelo si comparten clave (MLflow prohíbe cambiar un parámetro ya registrado); se resolvió prefijando los hiperparámetros del estimador (hp_), que además uniformó las columnas de comparación. Segundo: en Windows, mlflow run no puede construir el entorno declarado (requiere pyenv), por lo que se ejecuta con --env-manager local y el entorno declarado en python_env.yaml garantiza la reproducibilidad en plataformas que sí lo construyen; quedó documentado en el README.

Referencias

MLflow. (s.f.-a). MLflow Model Registry [Documentación]. https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (s.f.-b). MLflow Projects [Documentación]. https://mlflow.org/docs/latest/ml/projects/

MLflow. (s.f.-c). MLflow Tracking [Documentación]. https://mlflow.org/docs/latest/ml/tracking/

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). Introducing MLOps: How to scale machine learning in the enterprise. O'Reilly Media.

Lakshmanan, V., Robinson, S., & Munn, M. (2020). Machine learning design patterns: Solutions to common challenges in data preparation, model building, and MLOps. O'Reilly Media.